

CC1101-Funkanalyzer für Raspberry Pi & ioBroker

Eigenentwickelte, stabilitätsorientierte Lösung: bewährtes CC1101-Funk-Frontend + Node.js-Core-Dienst + nativer ioBroker-Adapter (Dienst + Bridge-Architektur)

CC1101 868 MHz

ATmega328P Radio-Frontend

Node.js / TypeScript Core

ioBroker-Adapter-Bridge

REST · WebSocket · MQTT

SQLite · InfluxDB · Grafana

Stability-First

Erstellt für: Silvio

Datum: 26. Juli 2026

Entwurfsentscheidung: Rohes CC1101-Frontend + komplette Eigenentwicklung der Analyse-Software

Leitprinzip: maximale Betriebsstabilität (24/7-Dauerbetrieb)

Software-Form: eigenständiger Core-Dienst + schlanke ioBroker-Adapter-Bridge

Status: Technisches Konzept / Designdokument

Inhaltsverzeichnis

1	Zielbild & Leitprinzipien
2	Gesamtarchitektur (3-Schichten-Modell)
3	Schicht 1 – Radio-Frontend (CC1101 + 328P)
4	Schicht 2 – Core-Dienst (Node.js/TypeScript)
5	Schicht 3 – ioBroker-Adapter-Bridge
6	Datenmodell & State-Baum
7	Stabilitäts- & Resilienzkonzept
8	Technologie-Stack & Repository-Struktur
9	Entwicklungs-Roadmap (Meilensteine)
10	Test- & Qualitätsstrategie
11	Risiken & Gegenmaßnahmen
12	Nächste Schritte & Quellen

1 Zielbild & Leitprinzipien

Ziel ist ein **eigenentwickelter, dauerhaft stabiler Homematic-Funkanalyser**, der auf einem Raspberry Pi läuft, den klassischen BidCoS/AskSin-Funkverkehr passiv mithört und die Ergebnisse sowohl in einem eigenständigen Dienst als auch nativ in ioBroker bereitstellt. Statt bestehende Software (AskSinAnalyzerXS) zu nutzen, wird die Analyseschicht neu und ioBroker-nativ entworfen – das Funk-Frontend bleibt jedoch bewusst die bewährte, unveränderte Sniffer-Hardware.

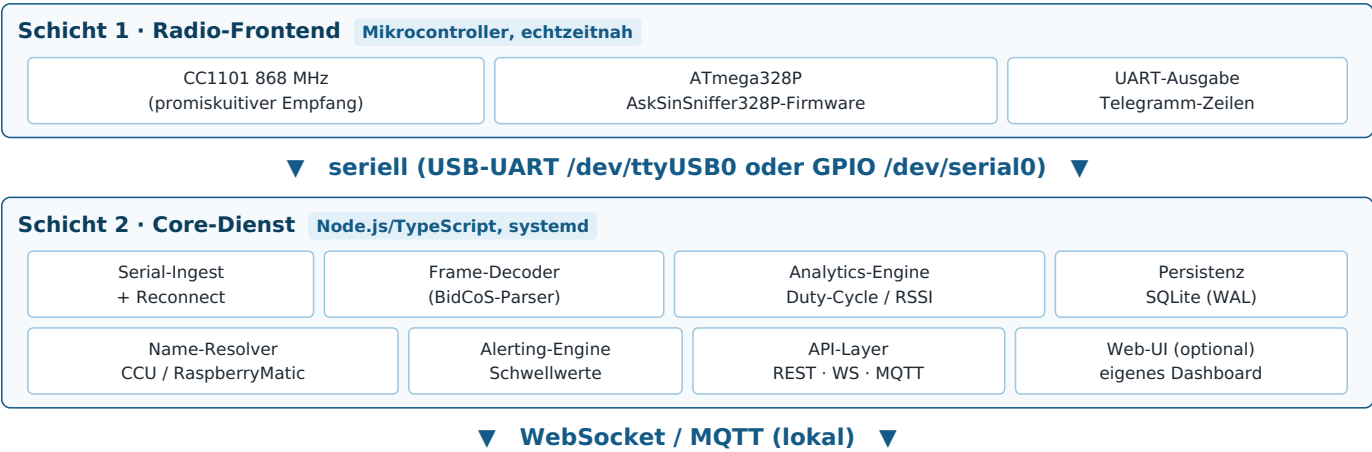
Leitprinzipien

- **Stabilität vor Funktionsumfang:** Jede Designentscheidung bevorzugt die robustere Variante.
- **Trennung von Echtzeit und Analyse:** zeitkritische Funkarbeit auf dediziertem Mikrocontroller, Auswertung auf dem Pi (Linux ist nicht echtzeitfähig).
- **Klare Schichten & Schnittstellen:** jede Schicht ist einzeln testbar und austauschbar.
- **Fehlertoleranz by design:** Reconnect, Backoff, Watchdog, begrenzte Puffer, crash-sichere Persistenz.
- **ioBroker-Best-Practices:** saubere Objekt-/State-Struktur, korrekte Rollen, sauberes `onUnload`.
- **Integrationsoffenheit:** der Core stellt REST + WebSocket + MQTT bereit – anders als XS.

Kern-Designentscheidung (Stabilität)

Das **Radio-Frontend** nutzt die bewährte **AskSinSniffer328P-Firmware unverändert**. Der ATmega328P erledigt ausschließlich die zeitkritische CC1101-/BidCoS-Empfangsarbeit und liefert fertige Telegramm-Zeilen seriell an den Pi. Der Pi berührt die Funk-Hardware nie direkt – das ist der robusteste denkbare Aufbau und entkoppelt die Analyse-Software vollständig von Funk-Timing.

2 Gesamtarchitektur (3-Schichten-Modell)



Schicht 3 · ioBroker-Adapter-Bridge ioBroker.asksinanalyser

Core-Client
(WS/MQTT-Abo)

State-/Objekt-Mapper
devices.*, info.*

Admin-Config
(jsonConfig)

VIS / Notifications
Scripte, Alarme

Die Bridge ist bewusst „dünn“: Sie hält keine Analyse-Logik, sondern spiegelt nur die Ergebnisse des Core-Dienstes in ioBroker-States. Fällt ioBroker aus, läuft die Analyse (und das Logging) im Core ungestört weiter – und umgekehrt.

Warum „Dienst + Bridge“ statt eines reinen Adapters?

Der Core-Dienst kapselt die dauerlaufende, sicherheitskritische Funk-/Analyse-Logik unabhängig vom ioBroker-Lebenszyklus (Adapter-Neustarts, js-controller-Updates). Die Bridge bleibt schlank und low-risk. Zusätzlich ist der Core auch ohne ioBroker (nur REST/MQTT/Grafana) nutzbar.

3 Schicht 1 - Radio-Frontend (CC1101 + 328P)

3.1 Aufbau

- **CC1101 (868 MHz)** im promiskuitiven Empfangsmodus – hört *alle* BidCoS-Telegramme, auch fremde, inkl. RSSI.
- **ATmega328P** (Pro Mini 3,3 V/8 MHz oder Nano) mit unveränderter `AskSinSniffer328P`-Firmware. Übernimmt Interrupt-getriebene Paketannahme (GDO0), Rahmen-Erkennung und serielle Ausgabe.
- Ausgabe je Telegramm als **eine serielle Textzeile** mit Feldern (u. a. Zähler/Zeit, RSSI, Länge, Msg-Flags, Msg-Type, Absender, Empfänger, Payload-Hex). Das exakte Zeilenformat wird 1:1 aus der Sketch-Quelle übernommen und im Parser abgebildet.

3.2 Serielle Anbindung - Empfehlung (Stabilität)

Option	Bewertung
USB-UART (empfohlen)	Eigener USB-Stack, entkoppelt von Pi-UART-Eigenheiten (serielle Konsole/Bluetooth), Sniffer räumlich frei platzierbar (bessere Antennenlage). <code>/dev/ttyUSB0</code> , per <code>udev</code> -Regel auf festen Namen (z. B. <code>/dev/asksin</code>) mappen.
GPIO-UART	Kompakt, aber UART muss konfiguriert werden (Konsole aus, BT umlegen). Antenne nah am Pi (potenzielle Störungen). Für Dauereinbau ok, für maximale Stabilität USB bevorzugen.

3.3 Stabilitätsmaßnahmen auf dieser Schicht

- Feste, saubere 3,3-V-Versorgung des CC1101; kurze SPI-Leitungen; korrekte 868-MHz-Antenne.
- **udev-Alias** für den seriellen Port, damit die Zuordnung bootfest ist.
- Optional **Hardware-Watchdog** auf dem 328P (Firmware-Watchdog) für Selbstheilung bei Hängern.
- Galvanische/räumliche Trennung von Störquellen (USB3-Ports, Schaltnetzteile).

4 Schicht 2 - Core-Dienst (Node.js/TypeScript)

Der Core ist ein eigenständiger, als `systemd`-Dienst laufender Prozess. Modularer Aufbau mit klar getrennten Verantwortlichkeiten:

Modul	Verantwortung
Serial-Ingest	Öffnet den Port, liest zeilenweise, erkennt Verbindungsabbrüche, reconnected mit Backoff, setzt Watchdog bei Funkstille.
Frame-Decoder	Parst die Telegramm-Zeile in ein typisiertes Objekt (siehe 6.1), validiert Länge/Plausibilität, verwirft Schrott robust.
Analytics-Engine	Duty-Cycle je Gerät (gleitendes 1-h-Fenster, Ringpuffer), RSSI-Statistik, Telegrammraten, Anomalien (Reboot-Loop, Dauersender, Stille).
Persistenz	SQLite im WAL-Modus für Events + Aggregate; optionaler InfluxDB-Writer für Grafana. Retention/Rotation konfigurierbar.
Name-Resolver	Bezieht Gerätenamen von CCU/RaspberryMatic (XML-API/ReGaHSS) oder FHEM; cached lokal, aktualisiert periodisch, funktioniert offline weiter.
Alerting-Engine	Wertet Schwellwerte aus, entprellt Alarme (Hysterese), publiziert Alarm-Events.
API-Layer	REST (Abfragen/Config), WebSocket (Live-Stream), MQTT-Publish (Integration/ioBroker-Bridge).
Web-UI (optional)	Eigenes leichtgewichtiges Dashboard (Live-Liste, Duty-Cycle-Ranking, RSSI) – kann später ergänzt werden.

4.1 Duty-Cycle-Berechnung (Kernlogik)

Pro Gerät wird die Sendezeit im gleitenden Stundenfenster kumuliert. Die Sendedauer je Telegramm wird aus Länge/Datenrate geschätzt (BidCoS $\approx$ 10 kbit/s). Umsetzung als **Ringpuffer** mit Zeitstempeln, damit Speicher fest begrenzt bleibt (keine unbeschränkt wachsenden Listen).

```
// Skizze: gleitender Duty-Cycle je Geräteadresse
onTelegram(t) {
  const airMs = estimateAirtime(t.length); // aus Länge + BidCoS-Datenrate
  ring.push({ ts: t.ts, airMs });           // Ringpuffer, feste Kapazität
  ring.dropOlderThan(t.ts - 3_600_000);    // 1-h-Fenster
  const dutyPct = ring.sumAirMs() / 3_600_000 * 100;
  publish(t.from, { dutyCycle: dutyPct, rssi: t.rssi, lastSeen: t.ts });
}
```

Hinweis: Der Wert ist eine belastbare Näherung für Trend/Alarm, kein amtlicher Regulierungsmesswert. Für die absolute Kalibrierung dient der Abgleich mit CCU-Duty-Cycle-Werten.

4.2 Namensauflösung

Der Resolver ruft die Geräteliste der CCU/RaspberryMatic ab (Seriennummer/Adresse $\rightarrow$ Klarname) und hält sie lokal vor. Bei CCU-Ausfall bleibt die letzte bekannte Zuordnung aktiv; die Analyse läuft adressbasiert ohnehin unabhängig weiter.

5 Schicht 3 - ioBroker-Adapter-Bridge

Der Adapter `ioBroker.asksinalyzer` abonniert den Core (WebSocket oder MQTT) und spiegelt die Ergebnisse in ioBroker-States. Er enthält bewusst **keine** Analyse-Logik.

5.1 Best-Practice-Konformität

- Objekte via `setObjectNotExistsAsync` / `extendObjectAsync` anlegen – nie `setObjectAsync` (überschreibt History-Einstellungen).
- Alle Werte vom „Gerät“ mit `ack: true` schreiben.
- `info.connection` (Rolle `indicator.connected`) über `instanceObjects` deklarieren und pflegen.
- Timer/Intervalle nur über `adapter.setTimeout` / `setInterval`; sauberes `onUnload` (alle Timer clearen, WS/MQTT schließen).
- Zugangsdaten (CCU) via `encryptedNative` / `protectedNative`.
- Vor Release: `adapter-check.iobroker.in` durchlaufen.

5.2 Admin-Konfiguration (jsonConfig)

- Core-Verbindung: Host/Port, Transport (WS/MQTT), Auth-Token.
- Optionaler Direktmodus: serieller Port (falls ohne separaten Core betrieben).
- Namensquelle: CCU-IP oder FHEM.
- Alarm-Schwellwerte (Duty-Cycle-%, RSSI-Grenze) – ergänzend zum Core.

6 Datenmodell & State-Baum

6.1 Telegramm-Objekt (intern)

```
interface Telegram {
  ts: number; // Empfangszeit (ms)
  rssi: number; // dBm
  length: number; // Bytes
  msgCounter: number; // BidCoS Msg-No
  flags: number; // BURST/WAKEUP/BIDI ...
  msgType: number; // z.B. 0x02=ACK, 0x10=INFO, 0x40=Taste, 0x41=Sensor ...
  from: string; // Absender-Adresse (3 Byte hex)
  to: string; // Empfänger-Adresse
  payloadHex: string; // Rohdaten
}
```

6.2 ioBroker State-Baum

State	Typ	Rolle	Einh.	Bedeutung
<code>info.connection</code>	boolean	<code>indicator.connected</code>	–	Core/Sniffer verbunden
<code>info.telegramsPerMinute</code>	number	value	1/min	Gesamt-Telegrammrates
<code>info.noiseFloor</code>	number	value	dBm	Grundrauschen

devices.<name>.dutyCycle	number	value	%	gleitender Duty-Cycle
devices.<name>.rssi	number	value.rssi	dBm	letzter RSSI
devices.<name>.lastSeen	number	date	-	letztes Telegramm
devices.<name>.msgPerHour	number	value	1/h	Telegrammrate
devices.<name>.address	string	info.address	-	BidCoS-Adresse
alarms.dutyCycleHigh	boolean	indicator.alarm	-	Duty-Cycle-Grenze überschritten
alarms.deviceSilent	boolean	indicator.alarm	-	erwartetes Gerät still

Rollen gemäß offizieller STATE_ROLES-Liste wählen; obige sind sinnvolle Vorschläge und werden vor Umsetzung final abgeglichen. Objekthierarchie: device → channel → state (Leaf).

7 Stabilitäts- & Resilienzkonzept

Da maximale Betriebsstabilität das Leitziel ist, hier die Maßnahmen gebündelt:

Aspekt	Maßnahme
Echtzeit-Entkopplung	Funk-Timing komplett auf dem 328P; der Pi macht nur Analyse.
Prozess-Resilienz	Core als systemd -Dienst mit Restart=always ; Adapter durch ioBroker-Prozessmodell überwacht. Beide unabhängig neustartbar.
Serielle Verbindung	Auto-Reconnect mit exponentiellem Backoff; udev-Alias für festen Port.
Funk-Watchdog	Bleiben Telegramme länger als X s aus → info.connection=false , Alarm, Reconnect-Versuch.
Speicherbegrenzung	Ringpuffer fester Größe für gleitende Fenster – kein unbegrenztes Wachstum.
Crash-sichere Persistenz	SQLite im WAL-Modus; periodische Aggregat-Flushes; Wiederanlauf ohne Datenverlust.
Backpressure	Bei Lastspitzen Eingangs-Queue mit Drop-Oldest statt Speicherüberlauf.
Zeitbasis	NTP am Pi sicherstellen – korrekte Duty-Cycle-Fenster hängen an der Uhr.
Entkopplung ioBroker	Analyse/Logging laufen im Core unabhängig weiter, auch wenn der Adapter oder js-controller neu startet.
Beobachtbarkeit	Strukturierte Logs, Health-Endpoint (REST /health), Selbst-Metriken (verarbeitete Telegramme, Fehlerzähler).
Datensicherung	Regelmäßiges Backup der SQLite-DB & Konfiguration; SD-Karten-Image oder besser Boot von SSD/USB.

8 Technologie-Stack & Repository-Struktur

Ebene	Technologie
Firmware	Arduino / C++ (bestehender AskSinSniffer328P-Sketch, unverändert)
Core-Dienst	Node.js + TypeScript; serialport , better-sqlite3 , fastify (REST) + ws , mqtt ; optional @influxdata/influxdb-client
ioBroker-Adapter	TypeScript, @iobroker/adapter-core , @iobroker/adapter-dev , jsonConfig-Admin
Dashboard/Logging	InfluxDB + Grafana (nativ oder via ioBroker-Adapter)
Betrieb	Raspberry Pi OS Lite 64-bit, systemd, udev, NTP

```
asksin-analyzer/
├─ firmware/                # Monorepo
├─ core/                    # AskSinSniffer328P (Referenz/Fork, unverändert)
├─ src/                    # Node.js/TS Core-Dienst
│  ├─ ingest/ (serial)
│  ├─ decode/ (BidCoS-Parser)
│  ├─ analytics/ (dutyCycle, anomalies)
│  ├─ persist/ (sqlite, influx)
│  ├─ resolve/ (ccu, fhem)
│  ├─ api/ (rest, ws, mqtt)
│  └─ test/ (Parser-Unit-Tests, Replays)
└─ ioBroker.asksinalyzer/  # Adapter-Bridge
   ├─ src/main.ts
   ├─ admin/ (jsonConfig)
   └─ io-package.json
```

9 Entwicklungs-Roadmap (Meilensteine)

MS	Ergebnis	Aufwand

M0	Frontend steht: Sniffer flashen, serielle Rohzeilen am Pi mitschneiden (Referenzdaten)	~1 Tag
M1	Parser-MVP: Frame-Decoder + Unit-Tests gegen aufgezeichnete Zeilen	~2-3 Tage
M2	Serial-Ingest mit Reconnect/Watchdog; robuster Dauerlauf	~1-2 Tage
M3	Analytics: Duty-Cycle, RSSI, Raten; Persistenz (SQLite/WAL)	~2-3 Tage
M4	Name-Resolver (CCU/RaspberryMatic)	~1-2 Tage
M5	API-Layer: REST + WebSocket + MQTT	~2 Tage
M6	ioBroker-Adapter-Bridge: States/Objekte, Admin-Config, info.connection	~2-4 Tage
M7	Alerting (Schwellwerte, Hysterese) + ioBroker-Benachrichtigungen	~1-2 Tage
M8	InfluxDB-Export + Grafana-Dashboards	~1-2 Tage
M9	Optionales eigenes Web-UI im Core	~2-3 Tage
M10	Härtung: Dauertest, Backups, systemd/udev, adapter-check, Doku	~2 Tage
Nutzbarer Kern (M0-M6)		ca. 11-17 Tage
Vollausbau (M0-M10)		ca. 17-26 Tage

Aufwände als Netto-Entwicklungszeit; MVP-Empfehlung: bis M3 kommen, dann früh am realen Funkverkehr validieren, bevor Bridge/Dashboard folgen.

10 Test- & Qualitätsstrategie

- **Aufzeichnung & Replay:** serielle Rohzeilen mitschneiden und als Fixtures in Unit-Tests einspielen – deterministische Decoder-Tests ohne Funk.
- **Parser-Unit-Tests** für alle bekannten Msg-Typen inkl. fehlerhafter/abgeschnittener Zeilen.
- **Langzeit-Soak-Test** (1-2 Wochen) zur Beobachtung von Speicher/Reconnect/Stabilität.
- **Plausibilitätsabgleich:** Duty-Cycle-Werte gegen CCU-Anzeige einzelner Geräte prüfen.
- **ioBroker adapter-check** und CI (Lint, Tests, Build) vor jedem Release.

11 Risiken & Gegenmaßnahmen

Risiko	Wahrsch.	Gegenmaßnahme
Seriellles Zeilenformat unklar/versioniert	mittel	Format aus Sketch verifizieren, Parser tolerant/versioniert auslegen, Replay-Tests.
Duty-Cycle-Schätzung ungenau	mittel	Airtime-Modell kalibrieren, gegen CCU abgleichen; als Trend/Alarm nutzen, nicht als Absolutmesswert.
Verpasste Telegramme bei Kollisionen	gering-mittel	Grenze eines Einzelempfängers dokumentieren; optional zweiter Sniffer.
USB-/UART-Instabilität	gering	udev-Alias, Reconnect/Backoff, Watchdog; USB-UART statt GPIO.
ioBroker-/js-controller-Updates	gering	Analyse im entkoppelten Core; Bridge dünn und schnell wiederherstellbar.
Nur BidCoS, kein HmIP	-	Bewusste Scope-Grenze; HmIP explizit außerhalb.

12 Nächste Schritte & Quellen

Empfohlene nächste Schritte

1. Sniffer-Hardware aufbauen/flashen (M0) und einige Stunden Rohdaten mitschneiden.
2. Seriellles Zeilenformat exakt aus der Sketch-Quelle dokumentieren.
3. Repo-Grundgerüst (Monorepo) anlegen; mit dem Parser-MVP (M1) starten.

Auf Wunsch liefere ich als Nächstes das **lauffähige Code-Grundgerüst** (Core-Dienst-Skelett mit Serial-Ingest-Stub + Parser-Gerüst und ioBroker-Adapter-Skelett mit State-Struktur) passend zu dieser Architektur.

Quellen & Referenzen

- Sniffer-Firmware & Wiki: github.com/jp112sdl/AskSinAnalyzer (AskSinSniffer328P)
- Referenz-Analyzer (Feature-Vergleich): github.com/psi-4ward/AskSinAnalyzerXS
- ioBroker Best Practices & State-Roles: github.com/ioBroker/ioBroker.repositories · STATE_ROLES.md
- ioBroker Adapter-Entwicklung: [@iobroker/adapter-core](https://github.com/ioBroker/adapter-core), [@iobroker/adapter-dev](https://github.com/ioBroker/adapter-dev), adapter-check.iobroker.in

Planungs-/Konzeptdokument. Verbindliche Feld-/Protokolldetails (serielles Zeilenformat, Msg-Typen, State-Rollen) sind vor der Umsetzung anhand der aktuellen Quellen final zu verifizieren.